https://github.com/ivaradkulkarni/ANNDL./tree/main/TA2

Reinforcement Learning for Stock Trading with LSTM & CNN Comparison

Course: Artificial Neural Networks & Deep Learning

Assignment: TA2 (Mini Project)

Members Name:

Harsh Gangapurkar A-21

Arin Jain A-39

Rushi Kulkarni A-55

Varad Kulkarni A-56

Dataset: S&P 500 Stock Data (Yahoo Finance)

Objective

The objective of this project is to apply Reinforcement Learning for stock trading and compare it with a Deep Learning model (LSTM).

The LSTM model is used to predict stock prices, while the Reinforcement Learning model is used to make trading decisions such as Buy, Sell, or Hold based on market conditions.

```
# Install if needed
# !pip install yfinance

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
import yfinance as yf
from tensorflow.keras import callbacks

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

sns.set_style('darkgrid')
plt.rcParams['figure.figsize'] = (10, 6)
```

Load Dataset

- In this step, I am loading stock price data for Apple (AAPL) using the yfinance library.

- I am using historical data from 2019 to 2024.

```
df = yf.download("AAPL", start="2019-01-01", end="2024-12-31")
df = df[['Close']]
df.dropna(inplace=True)

plt.plot(df)
plt.title("Stock Price")
plt.show()
```

```
/tmp/ipykernel_8585/3096274249.py:1: FutureWarning: YF.download() has
changed argument auto_adjust default to True
  df = yf.download("AAPL", start="2019-01-01", end="2024-12-31")
 [*********************100%***********************]  1 of 1 completed
```

```
print(df.head())
```

```
Price           Close
Ticker           AAPL
Date
2019-01-02  37.503723
2019-01-03  33.768082
```

```
2019-01-04  35.209621
2019-01-07  35.131252
2019-01-08  35.800961
```

```
df.size
```

```
1509
```

```
df.shape
```

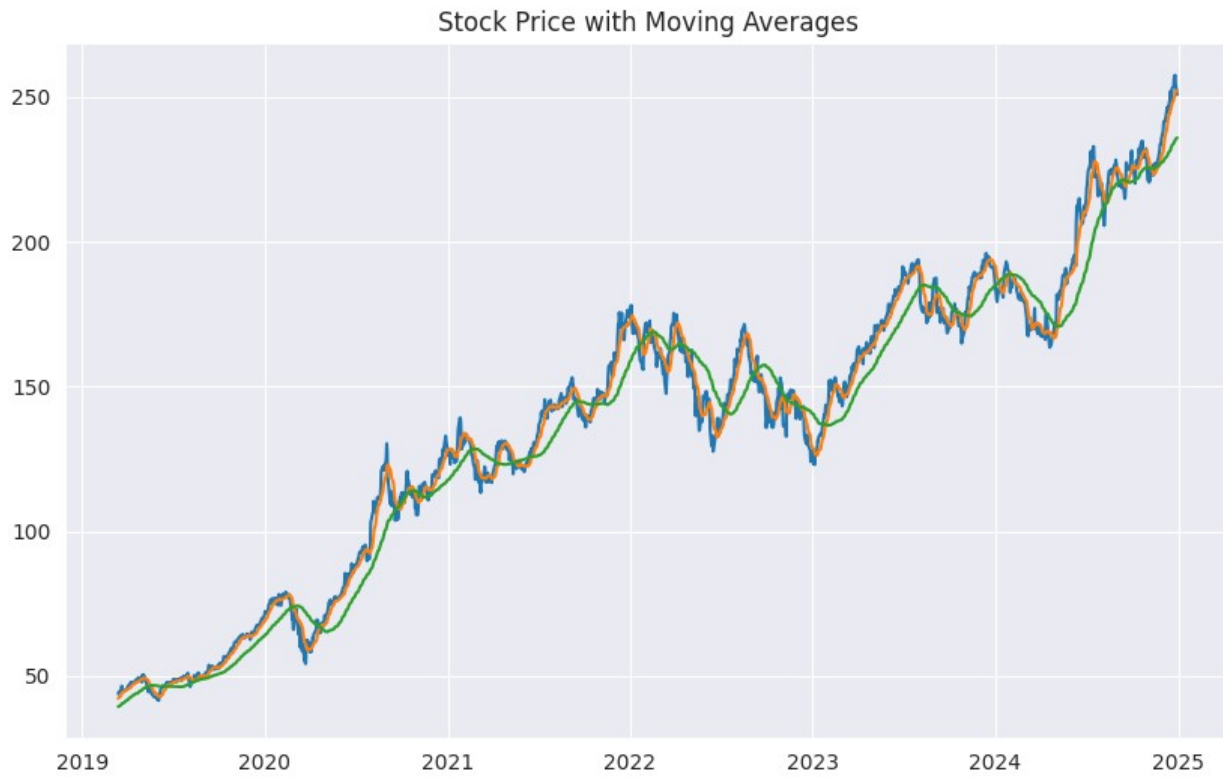
```
(1509, 1)
```

Feature Engineering

To improve the model, technical indicators such as moving averages and returns are added.

These features help the model capture trends and market behavior more effectively.

```
df['MA10'] = df['Close'].rolling(10).mean()
df['MA50'] = df['Close'].rolling(50).mean()
df['Returns'] = df['Close'].pct_change()

df.dropna(inplace=True)

plt.plot(df[['Close', 'MA10', 'MA50']])
plt.title("Stock Price with Moving Averages")
plt.show()
```




```
print(df.sample())

Price             Close        MA10       MA50   Returns
Ticker             AAPL
Date
2023-09-14  173.704239  179.030824  182.78043  0.008782
```

PART 1 — CNN MOdel

Convolutional Neural Network (CNN)

A 1D CNN model is used for time-series prediction.
CNN captures local patterns in data using convolution filters.

However, CNN is less effective than LSTM for long-term dependencies.

```
# CNN Model
cnn_model = tf.keras.Sequential([
    tf.keras.layers.Conv1D(filters=64, kernel_size=3,
activation='relu', input_shape=(X.shape[1], X.shape[2])),
    tf.keras.layers.MaxPooling1D(pool_size=2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(50, activation='relu'),
```

```
    tf.keras.layers.Dense(1)
])

cnn_model.compile(optimizer='adam', loss='mean_squared_error')

cnn_model.fit(X_train, y_train, epochs=10, batch_size=32)
```

```
Epoch 1/10

/usr/local/lib/python3.12/dist-packages/keras/src/layers/
convolutional/base_conv.py:113: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

34/34 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.0479
Epoch 2/10
34/34 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.0031
Epoch 3/10
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0020
Epoch 4/10
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0013
Epoch 5/10
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 8.6005e-04
Epoch 6/10
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 7.1144e-04
Epoch 7/10
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 6.4973e-04
Epoch 8/10
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 5.5286e-04
Epoch 9/10
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 5.2134e-04
Epoch 10/10
34/34 ━━━━━━━━━━━━━━━━━━━━ 1s 15ms/step - loss: 5.0093e-04

<keras.src.callbacks.history.History at 0x7970129fbda0>
```

```
cnn_pred = cnn_model.predict(X_test)

cnn_pred = scaler.inverse_transform(
    np.concatenate([cnn_pred, np.zeros((cnn_pred.shape[0], 3))],
axis=1)
)[:,0]

cnn_mape = np.mean(np.abs((y_test_actual - cnn_pred) / y_test_actual))
* 100

print("CNN MAPE:", cnn_mape)
```

```
9/9 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step
CNN MAPE: 4.397043497203859
```

CNN Model Evaluation

The CNN model achieved the following performance:

- Mean Absolute Percentage Error (MAPE): 4.39%

Observations:

- The CNN model provides moderate prediction accuracy.
- It is able to capture local patterns in stock price movements.
- However, it may not fully capture long-term dependencies in time-series data.

Conclusion:

The CNN model is useful for identifying short-term patterns in stock data, but its performance can be improved by using models that better handle sequential dependencies.

PART 2 — LSTM MODEL

Preprocessing

- Here, I normalize the stock prices using MinMaxScaler and convert the data into sequences.

- I am using a time step of 120, which means the model looks at the past 120 days to predict the next value.

```
features = df[['Close', 'MA10', 'MA50', 'Returns']]

scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(features)

time_step = 120

def create_sequences(data, time_step):
    X, y = [], []
    for i in range(time_step, len(data)):
        X.append(data[i-time_step:i])
        y.append(data[i, 0])  # predict Close price
    return np.array(X), np.array(y)

X, y = create_sequences(scaled_data, time_step)
```

Train-Test-Split

```
split = int(0.8 * len(X))

X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]
```

LSTM Model

I am using a stacked LSTM model to learn patterns from time-series data and predict future stock prices.

```
early_stopping_callback = callbacks.EarlyStopping(monitor='val_loss',
patience=5, restore_best_weights=True)

model = tf.keras.Sequential([
    tf.keras.layers.LSTM(64, return_sequences=True,
input_shape=(X.shape[1], X.shape[2])),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.LSTM(64),
    tf.keras.layers.Dense(1)
])

model.compile(optimizer='adam', loss='mean_squared_error')

history = model.fit(
    X_train, y_train,
    epochs=20,
    batch_size=32,
    validation_data=(X_test, y_test)
)
```

```
Epoch 1/20
34/34 ━━━━━━━━━━━━━━━━━━━━ 8s 134ms/step - loss: 0.0260 - val_loss:
0.0103
Epoch 2/20
34/34 ━━━━━━━━━━━━━━━━━━━━ 5s 153ms/step - loss: 0.0016 - val_loss:
0.0029
Epoch 3/20
34/34 ━━━━━━━━━━━━━━━━━━━━ 4s 124ms/step - loss: 0.0012 - val_loss:
0.0028
Epoch 4/20
34/34 ━━━━━━━━━━━━━━━━━━━━ 4s 122ms/step - loss: 0.0012 - val_loss:
0.0030
Epoch 5/20
34/34 ━━━━━━━━━━━━━━━━━━━━ 6s 160ms/step - loss: 0.0011 - val_loss:
0.0032
Epoch 6/20
34/34 ━━━━━━━━━━━━━━━━━━━━ 4s 123ms/step - loss: 0.0011 - val_loss:
0.0023
Epoch 7/20
34/34 ━━━━━━━━━━━━━━━━━━━━ 4s 122ms/step - loss: 9.1587e-04 -
```

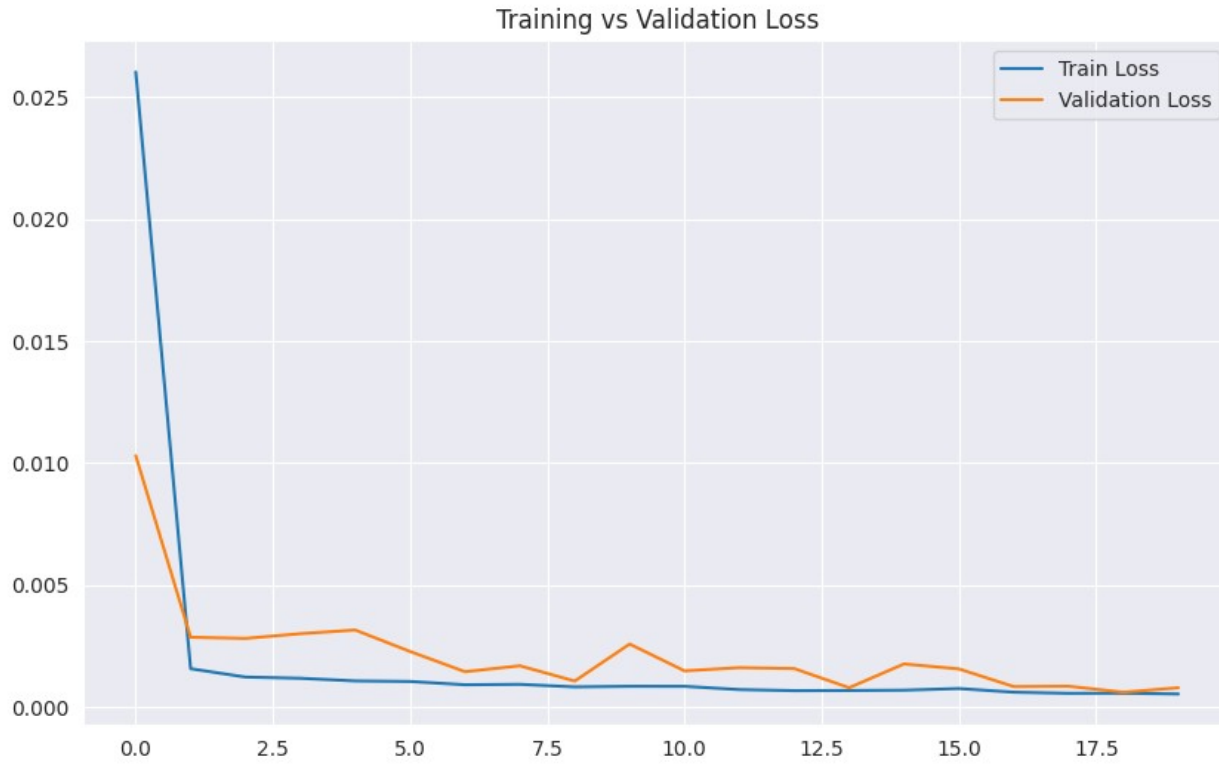
```
val_loss: 0.0015
Epoch 8/20
34/34 ━━━━━━━━━━━━━━━━━━━━ 6s 150ms/step - loss: 9.3267e-04 -
val_loss: 0.0017
Epoch 9/20
34/34 ━━━━━━━━━━━━━━━━━━━━ 4s 122ms/step - loss: 8.2806e-04 -
val_loss: 0.0011
Epoch 10/20
34/34 ━━━━━━━━━━━━━━━━━━━━ 5s 141ms/step - loss: 8.5227e-04 -
val_loss: 0.0026
Epoch 11/20
34/34 ━━━━━━━━━━━━━━━━━━━━ 5s 133ms/step - loss: 8.5077e-04 -
val_loss: 0.0015
Epoch 12/20
34/34 ━━━━━━━━━━━━━━━━━━━━ 4s 123ms/step - loss: 7.1801e-04 -
val_loss: 0.0016
Epoch 13/20
34/34 ━━━━━━━━━━━━━━━━━━━━ 6s 161ms/step - loss: 6.7369e-04 -
val_loss: 0.0016
Epoch 14/20
34/34 ━━━━━━━━━━━━━━━━━━━━ 4s 122ms/step - loss: 6.7970e-04 -
val_loss: 7.9365e-04
Epoch 15/20
34/34 ━━━━━━━━━━━━━━━━━━━━ 4s 121ms/step - loss: 6.9324e-04 -
val_loss: 0.0018
Epoch 16/20
34/34 ━━━━━━━━━━━━━━━━━━━━ 6s 147ms/step - loss: 7.6065e-04 -
val_loss: 0.0016
Epoch 17/20
34/34 ━━━━━━━━━━━━━━━━━━━━ 4s 122ms/step - loss: 6.1065e-04 -
val_loss: 8.4030e-04
Epoch 18/20
34/34 ━━━━━━━━━━━━━━━━━━━━ 5s 141ms/step - loss: 5.6439e-04 -
val_loss: 8.5454e-04
Epoch 19/20
34/34 ━━━━━━━━━━━━━━━━━━━━ 5s 135ms/step - loss: 5.6921e-04 -
val_loss: 6.0926e-04
Epoch 20/20
34/34 ━━━━━━━━━━━━━━━━━━━━ 5s 121ms/step - loss: 5.3865e-04 -
val_loss: 7.9385e-04
```

Training Performance

The training and validation loss are plotted to observe model learning.

```
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
```

```
plt.title("Training vs Validation Loss")
plt.show()
```



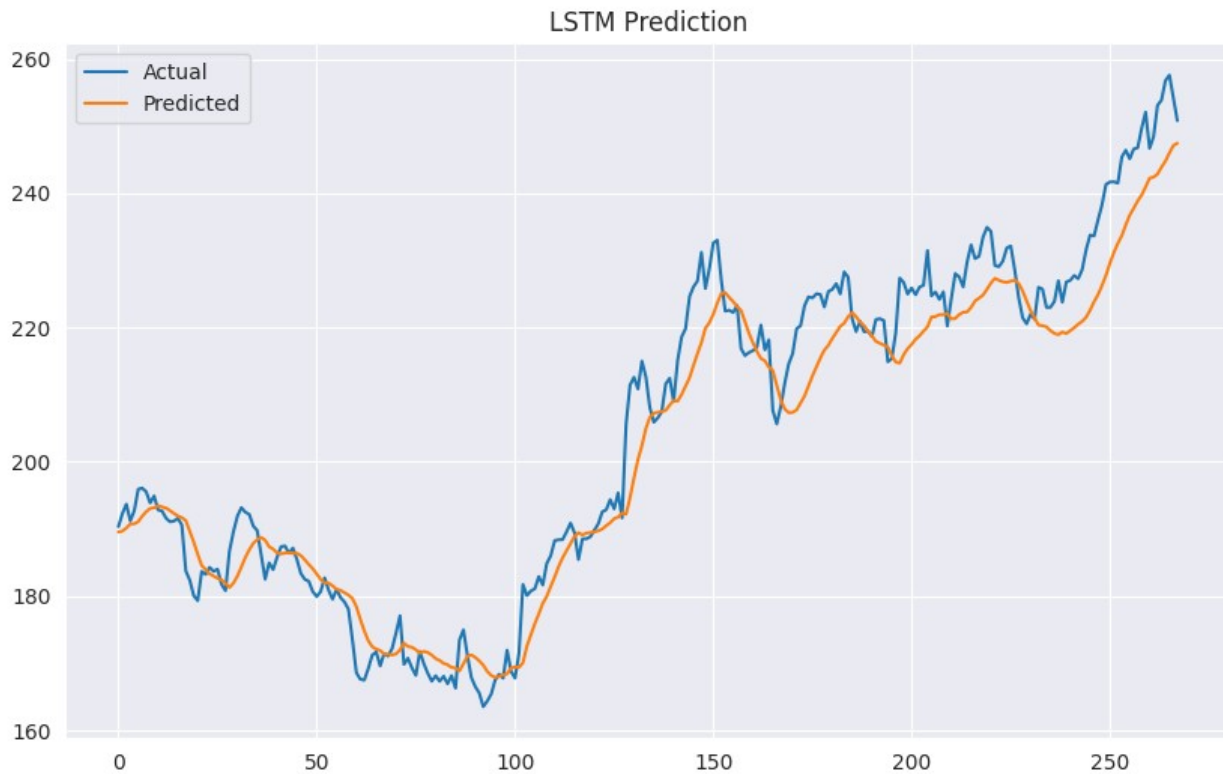
Prediction and Evaluation

In this step, I evaluate the model using MAE, RMSE, and MAPE.
These metrics help in understanding how accurate the predictions are.

```
pred = model.predict(X_test)

# Inverse scaling (only Close column)
pred = scaler.inverse_transform(
    np.concatenate([pred, np.zeros((pred.shape[0], 3))], axis=1)
)[:,0]

y_test_actual = scaler.inverse_transform(
    np.concatenate([y_test.reshape(-1,1), np.zeros((y_test.shape[0],
3))], axis=1)
)[:,0]

mae = mean_absolute_error(y_test_actual, pred)
rmse = np.sqrt(mean_squared_error(y_test_actual, pred))
mape = np.mean(np.abs((y_test_actual - pred) / y_test_actual)) * 100

print("MAE:", mae)
print("RMSE:", rmse)
```

```
print("MAPE:", mape)

plt.plot(y_test_actual, label="Actual")
plt.plot(pred, label="Predicted")
plt.legend()
plt.title("LSTM Prediction")
plt.show()
```

```
9/9 ━━━━━━━━━━━━━━━━━━━━ 1s 66ms/step
MAE: 4.7852921156738235
RMSE: 6.089281388291694
MAPE: 2.2610417654740416
```



Model Evaluation Results (LSTM)

The performance of the LSTM model is evaluated using MAE, RMSE, and MAPE:

- Mean Absolute Error (MAE): 4.79
- Root Mean Squared Error (RMSE): 6.09
- Mean Absolute Percentage Error (MAPE): 2.26%

Observations:

- The MAPE value of 2.26% indicates very high prediction accuracy with minimal error.

- The RMSE is slightly higher than MAE, suggesting a few larger deviations, but overall performance is strong.
- Compared to the CNN model (MAPE ≈ 4.39%), LSTM performs significantly better.
- This improvement is because LSTM can capture long-term dependencies in time-series data, whereas CNN mainly captures local patterns.

Conclusion:

The LSTM model provides more accurate predictions than CNN and is better suited for stock price forecasting.

PART 3 — RL

Reinforcement Learning Setup

In this part, I implement a simple Q-Learning model where the agent can take three actions:

- Hold
- Buy
- Sell

The reward is based on profit or loss.

```
prices = df['Close'].values.flatten()

actions = [0, 1, 2]  # Hold, Buy, Sell

q_table = np.zeros((len(prices), len(actions)))

alpha = 0.1
gamma = 0.9
epsilon = 0.1
```

Training the RL Agent

The agent learns by interacting with stock prices and updating the Q-table based on rewards.

```
for episode in range(10):
    total_reward = 0

    for t in range(len(prices)-1):

        if np.random.rand() < epsilon:
            action = np.random.choice(actions)
        else:
            action = np.argmax(q_table[t])

        price_today = prices[t]
```

```
        price_next = prices[t+1]

        if action == 1:
            reward = price_next - price_today
        elif action == 2:
            reward = price_today - price_next
        else:
            reward = 0

        q_table[t, action] += alpha * (
            reward + gamma * np.max(q_table[t+1]) - q_table[t, action]
        )

        total_reward += reward

    print(f"Episode {episode+1}, Reward: {total_reward}")
```

```
Episode 1, Reward: -31.071521759033203
Episode 2, Reward: 75.31652450561523
Episode 3, Reward: 146.6470947265625
Episode 4, Reward: 200.73294067382812
Episode 5, Reward: 237.8463897705078
Episode 6, Reward: 333.25426864624023
Episode 7, Reward: 416.88732528686523
Episode 8, Reward: 500.7210502624512
Episode 9, Reward: 566.293155670166
Episode 10, Reward: 657.8048782348633
```

Reinforcement Learning Results

The RL agent was trained over multiple episodes, and the reward increased consistently over time:

- Initial Reward (Episode 1): -31.07
- Final Reward (Episode 10): 657.80

This shows that the agent learned from experience and improved its trading strategy over time.

Testing RL Agent

Here, I test how much profit the RL agent can generate over time.

Understanding Reinforcement Learning

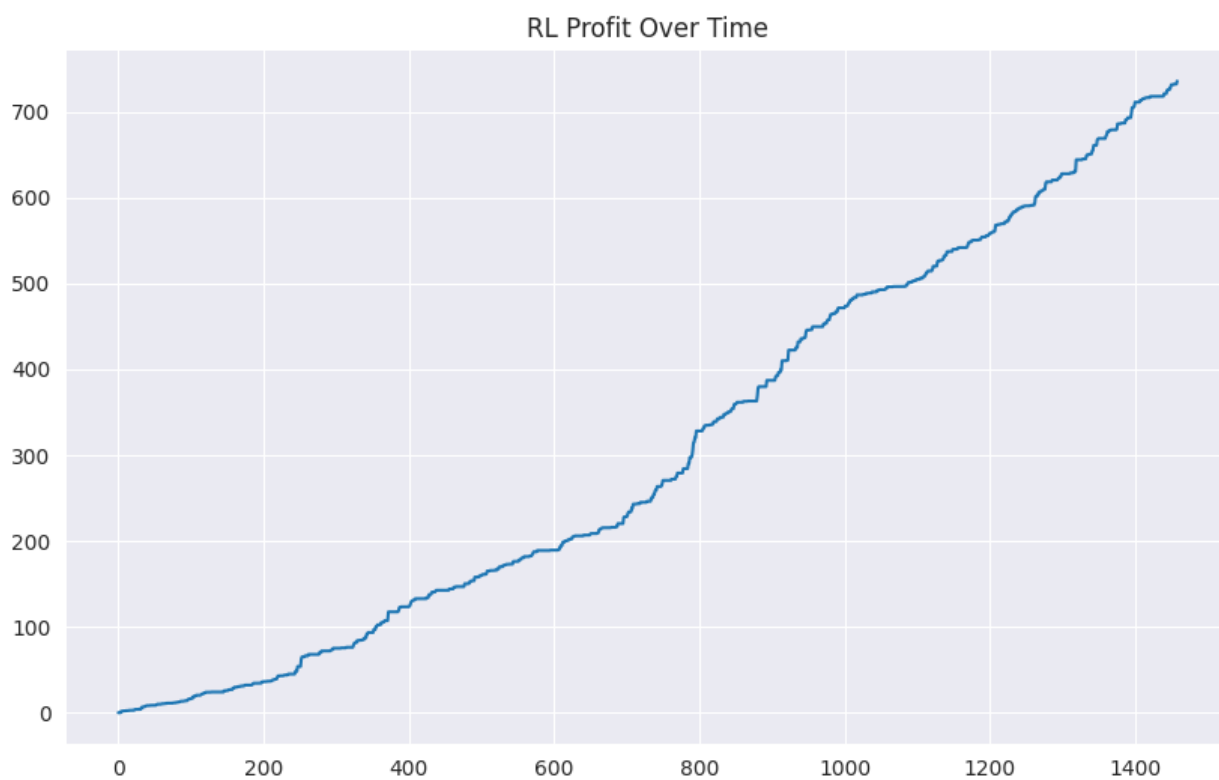
In this project, the RL agent acts like a trader that learns from experience.

At each time step, the agent chooses an action (Buy, Sell, or Hold) based on stock prices. It receives a reward based on profit or loss.

Over multiple episodes, the agent improves its decision-making strategy by maximizing rewards.

This allows the model to learn how to make profitable trading decisions rather than just predicting prices.

```
profit = 0
profits = []

for t in range(len(prices)-1):

    action = np.argmax(q_table[t])

    if action == 1:
        profit += prices[t+1] - prices[t]
    elif action == 2:
        profit += prices[t] - prices[t+1]

    profits.append(profit)

plt.plot(profits)
plt.title("RL Profit Over Time")
plt.show()

print("Total Profit:", profit)
```



```
Total Profit: 735.4794921875
```

Final Performance:

- Total Profit: 735.48

Observations:

- The increasing reward across episodes indicates successful learning.
- The agent improved its decision-making ability (Buy/Sell/Hold) over time.
- The final profit demonstrates that the RL model can effectively optimize trading strategies.

Model Comparison

Model	Type	Purpose	Performance	Strength	Limitation
CNN	Deep Learning	Pattern extraction	`MAPE ≈ 4.39%`	Captures local patterns	Weak for long-term dependencies
LSTM	Deep Learning	Time-series prediction	`MAPE ≈ 2.26%`	Learns sequential patterns	Slightly higher complexity
RL	Reinforcement Learning	Decision making	`Profit ≈ 735`	Optimizes actions for profit	Does not directly predict prices

Summary:

- LSTM performs best for stock price prediction due to its ability to capture temporal dependencies.
- CNN provides moderate performance by extracting local patterns but lacks long-term memory.
- RL focuses on maximizing profit through decision-making rather than prediction accuracy.

Since RL is a decision-making model, its performance is evaluated using cumulative profit instead of prediction error metrics like MAPE.

```
print(f"CNN MAPE: {cnn_mape}")
print(f"LSTM MAPE: {mape}")
print(f"RL Total Profit: {profit}")
```

```
CNN MAPE: 4.397043497203859
LSTM MAPE: 2.2610417654740416
RL Total Profit: 735.4794921875
```

Conclusion

In this project, I implemented CNN, LSTM, and Reinforcement Learning models for stock market analysis.

The LSTM model achieved the best prediction accuracy with a low error `(MAPE ≈ 2.26%)`, showing its effectiveness in time-series forecasting. The CNN model provided moderate performance `(MAPE ≈ 4.39%)` by capturing local patterns, but it was less effective than LSTM for long-term dependencies.

The Reinforcement Learning model, on the other hand, learned trading strategies over multiple episodes and generated a total profit of `735.48`.

This project highlights the difference between prediction and decision-making approaches. While CNN and LSTM focus on predicting stock prices, Reinforcement Learning focuses on maximizing profit through actions.

Overall, the comparison shows that LSTM is most suitable for stock prediction, CNN provides basic pattern learning, and RL is effective for decision-making in trading systems.